Game setup

Read before Game init; Web/Mobile have separate setup. Init guides; do not pre-read workflow/Webdev Skill.
Batch applicable Local/multiplayer guides with this page.

Game language

Use requested game language, else the user's Manus UI language (session default working language), then their
system language if known, then conversation language. Never use cloud sandbox locale.
New games use one language by default: pin this locale and disable inherited selectors and saved/OS/browser
auto-switching, overriding gameplay guides. Preserve existing-project localization; complete every exposed locale.
Requested Godot multilingual: template localization.
Completed single-language game: multilingual support is an optional follow-up; implement only on request.

Include the resolved game-content language and requested additional languages in init description requirements.
Model-authored Blueprint content uses the recorded session working language; the card's fixed UI follows the host UI locale.

Settle gameplay before initialization

For new Godot and Three.js games in Cloud or Local First, settle gameplay before `webdev.init_project`.
A request naming only a game or its dimension ("build a game", "build 2D game", "create 3D game") leaves
gameplay open; it is not delegation. Use the enrolled `game-dev-idea-generator` Skill's verified locator
and run its bundled `suggest-game-templates.mjs` once. Offer three sampled loops as concrete ideas through
the native question flow, plus None of the above/custom input, and wait for the user's direction before init.
Preserve requested 2D/3D; if unspecified, sample first and settle the dimension with the chosen idea before init.
Reuse a successful draw until asked to reroll. If the helper fails, ask for a custom direction; do not invent sampled results.

Preserve a supplied genre, core loop, remix or custom idea. Clear or explicitly delegated directions need no
extra clarification; when asked to choose, resolve a concrete idea before init. A title, dimension or task to
"define gameplay" is not settled gameplay. Include the selected idea and constraints in `description` for the
separate planner. Ideation creates no project, Blueprint or asset-generation approval. Continue existing projects
through their saved setup; do not restart them for ideation.

Remix names

Before init, give every remix (template, Showcase or supplied game) an original `title` and derived `name`.
Use a distinct requested title or invent one fitting the new brief. The selected source name is only a reference
in `description`, not the new title; changing case or appending "Remix"/"2" is insufficient.
Keep this identity through implementation and sharing, for both engines/all effort levels.

Select the engine

Use `gameEngine: "godot"` for 2D, `"threejs"` for 3D; honor explicit/existing engines. If unclear or template-conflicting,
ask once about 2D/3D before init, not the engine.
Omission defaults to Three.js; Godot must be explicit.

Three.js: `applicationKind: "game"`, `gameEngine: "threejs"`, name/title/full description; omit `gameStarter`
and Blueprint refs. Three-page Blueprint: gameplay, 3D asset preferences, online features.
Follow the Three.js Blueprint for approval and model permissions.
After approval, GET `webdev.config` `game/blueprint`; follow its next action, Web install/build/checkpoint/deployment
receipt and Local when local. No Godot tooling/release prep.
Before first Preview or `game/sharing` writes, read sharing; follow its Three.js preparation.
Historical previews: prepare after checkpoint; never block publish.

Start all games with server/database off: omit `features` or set both false. Describe requested online features; provisioning waits for
online approval and setup. Pending/cancelled choices are not approval.
Both engines' online multiplayer requires cloud computer selection

before init; reuse saved choices. It owns deployment/published-URL acceptance.
Next two sections: Godot only.

Direct start

Once genre/loop/direction are clear or explicitly delegated, call `webdev.init_project` with `applicationKind: "game"`, `gameEngine: "godot"`, name/title/full description, matching `gameStarter`.
Planner cannot see chat: include gameplay, references, recorded/delegated choices, visual inclusions/exclusions, effort, source restrictions and mockup consent.
No invented preferences/consent, separate Blueprint or production questions; the card settles unresolved choices.

Choose a matching starter, else `generic-multiplayer` online or `generic` offline; Fast prototype uses `generic`.
No starter questions. Omit Web-only template, template_id, framework, platform and port.
Init installs source/assets/provenance; checks tools; macOS may add missing Godot 4.7.2/Web templates, keeping
existing/custom installs. Init never starts Preview; never preview an untouched scaffold as the requested game.
Local First: managed services, no plain-local choice. Local owns directory authorization, Managed Git, environment and Session port. Use returned absolute paths and HOST/PORT; never guess Sandbox paths
or use other projects' files.

Starter selection

gameStarter	Current revision	Guide
`generic`	`game-generic-v56`	Guide
`platformer`	`game-platformer-v62`	Guide
`bullet-hell`	`game-bullet-hell-v52`	Guide
`vertical-platformer`	`game-vertical-platformer-v63`	Guide
`scroller`	`game-scroller-v63`	Guide
`td`	`game-td-v67`	Guide
`rts`	`game-rts-v63`	Guide
`trading-card`	`game-trading-card-v56`	Guide

`generic-multiplayer`	`game-generic-multiplayer-v38`	Guide
`puzzle`	`game-puzzle-v63`	Guide
`pinball`	`game-pinball-v9`	Guide

Use the receipt's revision/guide; legacy guides only if selected.
Never substitute/reapply templates.

After init

Implementation/sourcing require init, settled setup and requested concept approval.
Pending cards/notifications/scaffolds
are not approval. End the turn while waiting unless a newer message arrives; cards wake the session, so do not poll.
When `projectInitialized` is true, continue the same project; never initialize again.
Blueprint owns reopening, planning-unavailable handoff and setup recovery;
Godot Blueprint details owns concepts, session-owned coverage and legacy drafts.

Read and refresh saved decisions

After a Blueprint card response, GET `webdev.config` `game/blueprint` once; follow phase/revision/choices/next action.
GET never plans/approves/writes evidence. No separate Blueprint tool. Godot-only POST `game/blueprint/refresh` with `{expectedRevision}` for pending canonical coverage or Fast preparation; preserve
choices. Approved Fast installs Generic in the same project; report conflicts, never hand-replace files.
Approved Standard: no refresh for local evidence. Reconcile uncertain replies from saved state; never reinitialize,
invent keys or repeat planning. Session-owned coverage uses the question payload.

New messages while a card is pending

Run start closes this session's older unanswered setup, concept-review and generation-error cards. For a new
message during a run with a pending card, POST `game/blueprint/close` with `{blueprintId, expectedRevision}`. Follow
the message or, if approval wins, its receipt. Never close/reopen another session's Blueprint.